Multi-Tier Web Application Deployment Using Docker

1. Project Overview

1.1 The application

The **Expense Tracker** lets a user record and review personal expenses. A user can create, edit and delete expenses (amount, category, date and a note), filter them by date range, and see a total and a per-category breakdown on a dashboard.

Layer	Technology	Image
Presentation tier	React 18, TypeScript, Vite, Tailwind CSS and Redux Toolkit. Built into static files and served by Nginx.	`sawzidunn/expense-tracker-frontend:latest`
Application tier	ASP.NET Core 8 Minimal API (C#) with the MongoDB .NET driver.	`sawzidunn/expense-tracker-backend:latest`
Data tier	MongoDB 7	`mongo:7`
Orchestration	Docker Compose	Not an image. Defined in `docker-compose.yml`.

1.2 Repositories and images

Item	Location
Backend source code and Dockerfile	https://github.com/SawZiDunn/expense-tracker-backend
Frontend source code, Dockerfile and `nginx.conf`	https://github.com/SawZiDunn/expense-tracker-frontend
Deployment repository (Compose file only)	https://github.com/SawZiDunn/expense-tracker-deployment
Docker Hub images	`sawzidunn/expense-tracker-backend:latest` and `sawzidunn/expense-tracker-frontend:latest`

The deployment repository contains only `docker-compose.yml`, `.env.example`, `.gitignore` and `README.md`. It has no application source code. Because the application images are published on Docker Hub, anyone who clones it can start the whole system with a single command.

1.3 Running the project in one command

```
git clone https://github.com/SawZiDunn/expense-tracker-deployment
cd expense-tracker-deployment
docker compose up -d
```

Then open `http://localhost:3000` in a browser. Section 6 describes the steps in more detail.

2. Application Structure

2.1 Architecture

The application has **three tiers**. Each tier runs in its own container, and the database files are stored in a named volume.

Tier	Container	Image	Port
Presentation (frontend)	expense-tracker-frontend	sawzidunn/expense-tracker-frontend:latest	Container 80, published as host 3000
Application (backend)	expense-tracker-backend	sawzidunn/expense-tracker-backend:latest	Container 8080, published as host 8080
Data (database)	expense-tracker-mongo	mongo:7 Named volume: mongo-data (mounted at /data/db)	Container 27017, not published to the host

2.2 Why three containers instead of one

- The frontend, backend and database have separate image definitions and can be rebuilt or updated independently. For example, MongoDB can be upgraded without changing the application code.
- Each service has its own environment variables, logs and lifecycle. A crash in the backend does not by itself stop the frontend or database container.
- The database files are kept in a volume, separate from any container.
- The same configuration can be reproduced on another machine without installing Node.js, the .NET runtime and MongoDB on the host.

Separate containers do not automatically make a system highly available or secure. They provide isolation and clear deployment boundaries. Reliability still depends on configuration and on how the application handles failures.

3. Explanation of the Dockerfiles

Both Dockerfiles use multi-stage builds. The first stage contains the tools needed to compile the application. The final stage contains only what is needed to run it, so build tools and source code are not part of the final image.

3.1 Backend Dockerfile

```dockerfile
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app

# Copy csproj and restore dependencies
COPY *.csproj ./
RUN dotnet restore ExpenseTracker.csproj

# Copy everything else and build
COPY . ./
RUN dotnet publish ExpenseTracker.csproj -c Release -o out

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# Copy the built application from the build stage
COPY --from=build /app/out .

# Expose port (Render will provide PORT env variable)
EXPOSE 8080

# Run the application
ENTRYPOINT ["dotnet", "ExpenseTracker.dll"]
```

Port handling. The application reads the `PORT` environment variable to determine which port to listen on, defaulting to `5000` if it is not set. Dockerfile's `EXPOSE 8080` documents the intended port inside the container; it does not configure the application or expose the port to the host by itself. Docker Compose (we'll see below) maps the container port to the host using `8080:8080` (host port : container port), allowing access through `http://localhost:8080`.

3.2 Frontend Dockerfile

```dockerfile
# ---------- Build stage ----------
FROM node:20-alpine AS build

WORKDIR /app

# The backend API URL is baked into the static bundle at build time.
# Override it with: docker build --build-arg VITE_API_URL=https://api.example.com
ARG VITE_API_URL=http://localhost:5000
ENV VITE_API_URL=$VITE_API_URL

# Install dependencies
COPY package.json package-lock.json ./
RUN npm ci

# Copy the rest of the source and build the static bundle
COPY . .
RUN npm run build

# ---------- Runtime stage ----------
FROM nginx:1.27-alpine AS runtime

# SPA-friendly nginx config
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Serve the compiled assets
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Build-time configuration. Vite embeds `VITE_` variables into the generated JavaScript when the image is built. The browser receives static files, not a live connection to the container's environment. Changing `VITE_API_URL` later therefore has no effect on an image that is already built; the image must be rebuilt with the correct address.

3.3 Nginx configuration (nginx.conf)

expense-tracker-frontend > nginx.conf

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Compress text assets
    gzip on;
    gzip_min_length 1024;
    gzip_types text/plain text/css application/json application/javascript application/xml image/svg+xml;

    # Vite emits hashed filenames, so they can be cached forever
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        # A missing asset must 404 instead of falling back to index.html - otherwise the
        # browser receives HTML where it expects JavaScript and fails with a MIME error.
        try_files $uri =404;
    }

    # SPA fallback: unknown routes serve index.html so client-side routing works
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

- **Server block:** Nginx listens on port 80 and serves the files in `/usr/share/nginx/html`.
- **Compression:** `gzip` compresses text-based files such as CSS, JavaScript and JSON.
- **Single-page-app fallback:** any path that is not a real file returns `index.html`, so client-side routing keeps working when a page is reloaded.

4. Explanation of the docker-compose.yml File

4.1 The Compose file

docker-compose.yml

```
services:
    mongo:
        image: mongo:7
        container_name: expense-tracker-mongo
        restart: unless-stopped
        volumes:
            - mongo-data:/data/db
            - type: tmpfs
              target: /data/configdb
        healthcheck:
            test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
            interval: 10s
            timeout: 5s
            retries: 5
            start_period: 20s

    ▷Run Service
    backend:
        image: sawzidunn/expense-tracker-backend:latest
        container_name: expense-tracker-backend
        ports:
            - "8080:8080"
        environment:
            PORT: 8080
            ASPNETCORE_ENVIRONMENT: Production
            MONGODB_CONNECTION_STRING: ${MONGODB_CONNECTION_STRING:-mongodb://mongo:27017}
            FRONTEND_URL: http://localhost:3000
        depends_on:
            mongo:
                condition: service_healthy
        restart: unless-stopped

    ▷Run Service
    frontend:
        image: sawzidunn/expense-tracker-frontend:latest
        container_name: expense-tracker-frontend
        ports:
            - "3000:80"
        depends_on:
            - backend
        restart: unless-stopped

volumes:
    mongo-data:
```

4.2 Key settings

services. Each key under `services` describes one service. A service is a desired configuration; Compose creates a container from it. Here each service has exactly one container.

ports. The format is `HOST_PORT:CONTAINER_PORT`. Port `3000:80` forwards host port 3000 to Nginx in the frontend container, and `8080:8080` forwards host port 8080 to the backend. MongoDB has no `ports` entry, so it can be reached only from other containers on the Compose network.

environment and variable substitution. The Compose syntax `${VARIABLE:-default}` means: use the variable if it is set to a non-empty value (for example in a `.env` file); otherwise use the default. For `MONGODB_CONNECTION_STRING` the default is `mongodb://mongo:27017`, so by default the backend uses the local MongoDB container. It can be pointed at a cloud database such as MongoDB Atlas without editing the Compose file. The connection string does not name a database; the application selects `ExpenseTrackerDB` itself.

build args. `VITE_API_URL: ${VITE_API_URL:-http://localhost:8080}` passes the backend address to the frontend build, as explained in Section 4.2.

healthcheck and depends_on. The MongoDB health check runs `mongosh` inside the container and sends a `ping` command every 10 seconds. A probe fails if it takes longer than 5 seconds, and after 5 failed probes in a row the container is marked unhealthy. Failures during the first 20 seconds (`start_period`) are not counted, which gives MongoDB time to start. The backend uses `condition: service_healthy`, so Compose starts it only after the health check passes.

4.3 Volumes and data persistence

MongoDB stores its data files in `/data/db`. The named volume `mongo-data` is mounted there. It is managed by Docker and exists independently of the container, so the container can be removed and recreated without losing the database files.

The MongoDB image also declares `/data/configdb` as a volume path. If that path is not mounted explicitly, Docker creates an anonymous volume for it each time a container is created, which can leave unnamed volumes behind. The Compose file mounts a `tmpfs` at `/data/configdb` instead. A tmpfs is temporary, memory-backed storage that is not kept when the container stops.

Running docker compose down -v will remove the volume mongo-data, though.

A volume is storage, not a running server. When the MongoDB container is stopped, the data files remain but nothing can query them. A database client on the host also needs the database port to be published; this project does not publish it. For direct access from the host, a mapping such as `127.0.0.1:27017:27017` could be added to the `mongo` service, which limits access to the local machine.

5. Building and Running the Containers

5.1 Requirements

- Docker Engine with Docker Compose v2 (the `docker compose` command).
- Internet access to pull the images from Docker Hub.
- Free host ports 3000 and 8080.

5.2 Method A: Run the Published Images

The deployment repository contains the Docker Compose configuration but does not require the frontend or backend source code. It uses the images published on Docker Hub.

```
git clone https://github.com/SawZiDunn/expense-tracker-deployment

cd expense-tracker-deployment


# Optional: create a .env file to override default settings

docker compose pull

docker compose up -d

docker compose ps
```

Compose downloads the published images, creates the required network and volume, starts MongoDB, waits for its health check, and then starts the backend and frontend. The `.env` file is optional because the default configuration uses a local MongoDB container.

Open `http://localhost:3000` in a browser to access the application.

5.3 Method B: Build the Images from Source

This method is useful for development or when testing source-code changes. In addition to the deployment repository, clone the frontend and backend repositories into the deployment folder.

```
git clone https://github.com/SawZiDunn/expense-tracker-deployment

cd expense-tracker-deployment


git clone https://github.com/SawZiDunn/expense-tracker-backend

git clone https://github.com/SawZiDunn/expense-tracker-frontend


docker compose -f docker-compose.yml -f docker-compose.build.yml up --build -d
```

The additional Compose file provides the build configuration. Docker builds the frontend and backend images from their respective Dockerfiles before starting the services. The default environment settings can be used without creating a `.env` file.

To override configuration, create a `.env` file in the deployment repository. For example, `MONGODB_CONNECTION_STRING` can select a cloud database, while `VITE_API_URL` can override the frontend API URL used during the build.

Open `http://localhost:3000` in a browser to access the application.

5.4 Publishing the images

The images named in the Compose file are published to Docker Hub under the `sawzidunn` account. After building them, they can be pushed with:

```
docker login
docker compose build
docker compose push
```

Because `VITE_API_URL` is embedded at build time, the frontend image must be built with an address that the user's browser can reach. The default `http://localhost:8080` works when the browser and Docker run on the same machine.

6. Relation to Operating System Concepts

6.1 Process isolation

A container is a group of ordinary Linux processes that are isolated by kernel features. **Namespaces** give each container its own view of selected resources: process IDs, network interfaces, mount points and host name. **Control groups (cgroups)** account for resources such as CPU and memory and can limit them when limits are configured.

Containers do not run their own kernel. All containers share the host's Linux kernel, while a virtual machine normally runs a separate guest operating system with its own kernel. In this project the frontend, backend and database run as separate processes in separate namespaces, each with its own configuration. `docker compose top` shows one process tree per container.

	Container	**Virtual machine**
Kernel	Shared with the host	Separate guest kernel
Isolation mechanism	Namespaces and cgroups	Hypervisor and virtual hardware
Typical start-up and size	Starts like a process; images are small	Boots an operating system; images are larger

6.2 File systems and persistent storage

An image is built from read-only layers. Each Dockerfile instruction can create a layer, and Docker caches unchanged layers. This is why the Dockerfiles copy `package.json` and the `.csproj` file before the rest of the source: the dependency layers stay cached while the application code changes.

A running container adds a writable layer on top of the image layers. That layer disappears with the container, so it is not suitable for data that must survive. The named volume `mongo-data` stores the database files outside the container's lifecycle. The `tmpfs` mount at `/data/configdb` shows a third option: memory-backed storage that is deliberately temporary.

6.3 Networking

Each container has its own network namespace. Compose connects the services to a shared network with built-in name resolution, so the backend can reach the database with the host name `mongo`. The browser is outside this network. It reaches the containers only through ports that Docker publishes on the host: `3000` for the frontend and `8080` for the backend. This is why the backend uses

`mongodb://mongo:27017` while the frontend is configured with `http://localhost:8080`. The database port is not published, so it is not reachable from outside the Compose network.

6.4 Process synchronisation and readiness

The backend needs the database before it can work. Starting a process is not the same as the process being ready, so the health check lets Compose observe readiness and `condition: service_healthy` makes the backend wait for it. This is a practical example of ordering dependent processes.

6.5 Environment variables and configuration

Environment variables are part of a process's environment and are passed to the process when it starts. The backend reads `PORT`, `MONGODB_CONNECTION_STRING` and `FRONTEND_URL` this way. The frontend is different: its configuration is fixed at build time, because the browser runs static files. Understanding when a value is read (build time or run time) explains why `VITE_API_URL` requires a rebuild.

7. Problems Encountered and Solutions

Problem or question	Explanation	Solution
How can the whole application run without a cloud database account?	A database can run in its own container.	Run MongoDB 7 as the `mongo` service with a named volume. Atlas remains possible through `MONGODB_CONNECTION_STRING`.
Does a volume mean the database is running? Why can a database tool not connect when the container is stopped?	The volume stores files. The MongoDB process serves requests. The database port is also not published to the host.	Start the MongoDB container. For access from the host, a port mapping such as `127.0.0.1:27017:27017` can be added.
What is the anonymous volume at `/data/configdb`?	The MongoDB image declares this path as a volume. Without an explicit mount, Docker creates an unnamed volume for it.	Mount a `tmpfs` at `/data/configdb`, because the data there does not need to persist.
Why does changing `VITE_API_URL` not change the running frontend?	Vite embeds the value into the JavaScript at build time.	Rebuild the frontend image with the correct `VITE_API_URL`.

8. How AI Was Used in the Project

AI was used as a development and learning assistant for Expense Tracker, not as a runtime component. I used an AI assistant (OpenCode) to help with the following:

- Understanding Docker Compose service definitions, build contexts, images and containers.
- Reasoning about database persistence and the differences between named volumes, anonymous volumes and tmpfs.
- Understanding how containers communicate through service names and published host ports.
- Understanding how Vite handles environment variables at build time.
- Organising troubleshooting notes and operating system concepts, and drafting and structuring this report.

9. Conclusion

The Expense Tracker was deployed as a three-tier system in three containers: an Nginx-served React frontend, an ASP.NET Core backend and a MongoDB database. Multi-stage Dockerfiles produce small runtime images, and a single Compose file connects the services, publishes the right ports, passes configuration, waits for the database to be healthy and keeps the data in a named volume. Because the images are published on Docker Hub, the deployment repository can start the whole system with one command.

The project also shows several operating system ideas in practice: process isolation with namespaces and cgroups, layered and persistent file systems, container networking, readiness and synchronisation, and the difference between build-time and run-time configuration.